

UNIVERSIDAD DE EL SALVADOR
FACULTAD MULTIDISCIPLINARIA ORIENTAL
DEPARTAMENTO DE INGENIERÍA Y ARQUITECTURA
INGENIERIA DE SISTEMAS INFORMÁTICOS



ASIGNATURA:

BASES DE DATOS

PROYECTO FINAL:

SISTEMA DE CONTROL DE ASISTENCIA, NOTAS Y SEGUIMIENTO
ESTUDIANTIL – COMPROBANTE

DOCENTE:

ING. VÁSQUEZ MARTÍNEZ JOSÉ GUADALUPE

ESTUDIANTES:

RETANA MEDINA, ALEXANDER ANTOLINO - RM21077

MARTÍNEZ JOVEL, JOSÉ ANTONIO - MJ21008

VÁSQUEZ CANALES, EVER SAMUEL - VC21033

CIUDAD UNIVERSITARIA ORIENTAL, 23 DE JUNIO DEL 2026.

1	SELECT ia.id_inscripcion, ia.id_matricula, ia.promedio_t1, ia.promedio_t2, ia.promedio_final					
2	FROM inscripciones_asignaturas ia					
3	WHERE ia.id_matricula = 8					
4	ORDER BY ia.id_inscripcion;					

	id_inscripcion [PK] integer	id_matricula integer	promedio_t1 numeric (4,2)	promedio_t2 numeric (4,2)	promedio_final numeric (4,2)
1	36	8	9.50	9.75	9.63
2	37	8	9.50	9.75	9.63
3	38	8	9.50	9.75	9.63
4	39	8	9.50	9.75	9.63
5	40	8	9.50	9.75	9.63

Se consulta el estado actual de los promedios del estudiante con matrícula 8 (David Crespo), antes de realizar cualquier modificación. Se observa que sus promedios reflejan un desempeño alto, consistente con los datos de la guía de prueba.

1	UPDATE notas					
2	SET nota = 2.00					
3	WHERE id_inscripcion = (SELECT MIN(id_inscripcion) FROM inscripciones_asignaturas WHERE id_matricula = 8)					
4	AND id_evaluacion = (					
5	SELECT id_evaluacion FROM evaluaciones					
6	WHERE id_asignacion_docente = (					
7	SELECT id_asignacion_docente FROM inscripciones_asignaturas					
8	WHERE id_inscripcion = (SELECT MIN(id_inscripcion) FROM inscripciones_asignaturas WHERE id_matricula = 8)					
9	)					
10	AND trimestre = 1					
11	ORDER BY id_evaluacion LIMIT 1					
12	);					

	id_inscripcion [PK] integer	id_matricula integer	promedio_t1 numeric (4,2)	promedio_t2 numeric (4,2)	promedio_final numeric (4,2)
1	36	8	6.50	9.75	8.13
2	37	8	9.50	9.75	9.63
3	38	8	9.50	9.75	9.63
4	39	8	9.50	9.75	9.63
5	40	8	9.50	9.75	9.63

Se reduce intencionalmente una de las notas del Trimestre 1 del estudiante a un valor reprobatorio (2.00), con el fin de forzar la activación del trigger trg_actualizar_promedio.

1	SELECT ia.id_inscripcion, ia.id_matricula, ia.promedio_t1, ia.promedio_t2, ia.promedio_final					
2	FROM inscripciones_asignaturas ia					
3	WHERE ia.id_matricula = 8					
4	ORDER BY ia.id_inscripcion;					

	id_inscripcion [PK] integer	id_matricula integer	promedio_t1 numeric (4,2)	promedio_t2 numeric (4,2)	promedio_final numeric (4,2)
1	36	8	6.50	9.75	8.13
2	37	8	9.50	9.75	9.63
3	38	8	9.50	9.75	9.63
4	39	8	9.50	9.75	9.63
5	40	8	9.50	9.75	9.63

Al consultar nuevamente la inscripción, se confirma que el trigger recalculó automáticamente promedio_t1 y promedio_final sin necesidad de ejecutar ninguna actualización manual adicional, demostrando que la lógica de cálculo ponderado funciona correctamente.

1	SELECT	a.id_inscripcion	,	a.tipo_alerta	,	a.estado_alerta	,	a.descripcion
2	FROM	alertas_academicas	a					
3	JOIN	inscripciones_asignaturas	ia	ON	ia.id_inscripcion	=	a.id_inscripcion	
4	WHERE	ia.id_matricula	=	3	AND	a.tipo_alerta	=	'inasistencia_excesiva'
5	ORDER	BY	a.fecha_alerta	DESC				

	id_inscripcion	tipo_alerta	estado_alerta	descripcion
	integer	character varying (30)	character varying (15)	text
1	11	inasistencia_excesiva	activa	Alerta de Asistencia: El estudiante supera el 25% de ausencias (50.00% de ausencias en 4 clas...
2	11	inasistencia_excesiva	resuelta	Alerta de Asistencia: El estudiante supera el 25% de ausencias (50.00% de ausencias en 2 clas...
3	13	inasistencia_excesiva	resuelta	Inasistencias justificadas por viaje familiar autorizado por dirección.

Se consulta la alerta de tipo inasistencia_excesiva correspondiente a la matrícula 3, generada previamente por el patrón de ausencias cargado en la semilla de datos. La alerta se encuentra en estado activa.

1	UPDATE	asistencias
2	SET	tipo_asistencia = 'justificada', observacion = 'Justificada con constancia médica'
3	WHERE	id_inscripcion = (
4		SELECT ia.id_inscripcion FROM inscripciones_asignaturas ia
5		JOIN asignaciones_docentes ad ON ad.id_asignacion_docente = ia.id_asignacion_docente
6		WHERE ia.id_matricula = 3 AND ad.id_materia = 21
7		)
8	AND	tipo_asistencia = 'ausente';

Data Output	Messages	Notifications
UPDATE 2		
Query returned successfully in 170 msec.		

Se actualizan las inasistencias registradas para esta matrícula, cambiando su tipo de ausente a justificada, simulando la corrección de un registro tras la presentación de una constancia médica.

1	SELECT	a.id_inscripcion	,	a.tipo_alerta	,	a.estado_alerta	,	a.descripcion	,	a.fecha_alerta
2	FROM	alertas_academicas	a							
3	JOIN	inscripciones_asignaturas	ia	ON	ia.id_inscripcion	=	a.id_inscripcion			
4	WHERE	ia.id_matricula	=	3	AND	a.tipo_alerta	=	'inasistencia_excesiva'		
5	ORDER	BY	a.fecha_alerta	DESC						

	id_inscripcion	tipo_alerta	estado_alerta	descripcion	fecha_alerta
	integer	character varying (30)	character varying (15)	text	timestamp without time zone
1	11	inasistencia_excesiva	resuelta	Alerta de Asistencia: El estudiante supera el 25% de ausencias (50.00% de ausencias en 4 clas...	2026-06-20 21:29:33.63426
2	11	inasistencia_excesiva	resuelta	Alerta de Asistencia: El estudiante supera el 25% de ausencias (50.00% de ausencias en 2 clas...	2026-06-20 20:00:51.51900
3	13	inasistencia_excesiva	resuelta	Inasistencias justificadas por viaje familiar autorizado por dirección.	2026-02-12 09:45:00

Al consultar nuevamente la alerta, se observa que su estado cambió de activa a resuelta sin ejecutar ningún UPDATE directo sobre la tabla alertas_academicas. Esto confirma que el trigger trg_actualizar_alertas_asistencia evalúa el porcentaje de ausencias en cada operación y resuelve la alerta automáticamente cuando el estudiante deja de superar el límite permitido (25%).

```
1 UPDATE notas SET nota = 8.75
2 WHERE id_inscripcion = 2 AND id_evaluacion = (
3     SELECT id_evaluacion FROM evaluaciones
4     WHERE id_asignacion_docente = (SELECT id_asignacion_docente FROM inscripciones_asignaturas WHERE id_inscripcion = 2)
5     AND trimestre = 1
6     ORDER BY id_evaluacion LIMIT 1
7 );
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 71 msec.

Se modifica manualmente una nota de la inscripción 2 mediante una instrucción UPDATE directa sobre la tabla notas.

```
1 SELECT * FROM auditoria_notas
2 WHERE id_inscripcion = 2
3 ORDER BY fecha_hora DESC
4 LIMIT 5;
```

Data Output Messages Notifications

	id_auditoria [PK] integer	id_nota integer	id_inscripcion integer	id_evaluacion integer	operacion character varying (10)	nota_anterior numeric (4,2)	nota_nueva numeric (4,2)	usuario_bd character varying (80)	fecha_hora timestamp without time zone
1	1213	6	2	6	UPDATE	8.75	8.75	postgres	2026-06-20 23:22:57.56064
2	1211	6	2	6	UPDATE	9.00	8.75	postgres	2026-06-20 21:35:28.695383
3	1210	6	2	6	UPDATE	9.00	9.00	postgres	2026-06-20 21:35:13.187036
4	1209	6	2	6	UPDATE	8.75	9.00	postgres	2026-06-20 21:35:05.848386
5	1203	6	2	6	UPDATE	8.50	8.75	postgres	2026-06-20 21:29:45.29889

Al consultar la tabla auditoria_notas, se confirma que el trigger trg_auditoria_notas registró automáticamente la operación, incluyendo el valor anterior (nota_anterior) y el nuevo valor (nota_nueva), además del usuario y la fecha exacta del cambio.

```
1 SELECT * FROM auditoria_notas
2 WHERE id_inscripcion = 2
3 ORDER BY fecha_hora DESC
4 LIMIT 5;
5 CALL registrar_nota(2, 6, 9.00);
```

Data Output Messages Notifications

CALL

Query returned successfully in 163 msec.

Se ejecuta el procedimiento almacenado registrar_nota, que internamente realiza un UPSERT sobre la misma nota, esta vez sin usar una instrucción UPDATE escrita manualmente.

```
1 SELECT * FROM auditoria_notas
2 WHERE id_inscripcion = 2
3 ORDER BY fecha_hora DESC
4 LIMIT 5;
```

	id_auditoria [PK] integer	id_nota integer	id_inscripcion integer	id_evaluacion integer	operacion character varying (10)	nota_anterior numeric (4,2)	nota_nueva numeric (4,2)	usuario_bd character varying (80)	fecha_hora timestamp without time zone
1	1214	6	2	6	UPDATE	8.75	9.00	postgres	2026-06-20 23:25:50.560705
2	1213	6	2	6	UPDATE	8.75	8.75	postgres	2026-06-20 23:22:57.56064
3	1211	6	2	6	UPDATE	9.00	8.75	postgres	2026-06-20 21:35:28.695383
4	1210	6	2	6	UPDATE	9.00	9.00	postgres	2026-06-20 21:35:13.187036
5	1209	6	2	6	UPDATE	8.75	9.00	postgres	2026-06-20 21:35:05.848386

Se confirma que la bitácora de auditoría también registró el cambio realizado a través del procedimiento, demostrando que el trigger funciona de manera consistente sin importar el método utilizado para modificar la nota (UPDATE manual o procedimiento almacenado), lo cual garantiza trazabilidad completa del sistema.

```
1 DELETE FROM inscripciones_asignaturas
2 WHERE id_inscripcion = (SELECT MIN(id_inscripcion) FROM inscripciones_asignaturas WHERE id_matricula = 4);
```

DELETE 1

Query returned successfully in 146 msec.

Se elimina la inscripción correspondiente a la matrícula 4 mediante una instrucción DELETE sobre la tabla inscripciones_asignaturas.

```
1 SELECT * FROM respaldo_inscripciones
2 WHERE id_matricula = 4
3 ORDER BY fecha_eliminacion DESC;
```

	id_respaldo [PK] integer	id_inscripcion integer	id_matricula integer	id_asignacion_docente integer	promedio_t1 numeric (4,2)	promedio_t2 numeric (4,2)	promedio_t3 numeric (4,2)	promedio_final numeric (4,2)	estado character varying (15)	eliminado character
1	2	17	4	2	6.15	6.25	[null]	6.20	activa	postgres
2	1	16	4	1	6.15	6.25	[null]	6.20	activa	postgres

Al consultar la tabla respaldo_inscripciones, se confirma que, antes de completarse la eliminación, el trigger trg_respaldo_inscripcion copió automáticamente los datos del registro eliminado (incluyendo sus promedios y estado), preservando así la información histórica del estudiante incluso después de su eliminación definitiva.

```
1 INSERT INTO horarios (id_asignacion_docente, dia_semana, hora_inicio, hora_fin, aula)
2 VALUES (6, 1, '07:00:00', '08:30:00', 'Aula 1B');
```

ERROR: Choque de Horario: El docente Alejandra Alvarez ya tiene asignada una clase en el día y horario solicitado (07:00:00 - 08:30:00).
CONTEXT: función PL/pgSQL fn_validar_horario() en la línea 30 en RAISE

SQL state: P0001

Se intenta registrar un nuevo horario para la docente Alejandra Alvarez en un día y hora donde ya tiene una clase asignada en otra sección. El sistema rechaza la operación mostrando el mensaje de error definido en el trigger trg_validar_horario: *"Choque de Horario: El docente ya tiene asignada una clase en el día y horario solicitado"*. Esto demuestra que la base de datos protege la integridad del horario escolar, impidiendo que un mismo docente quede asignado a dos clases simultáneas, incluso si el error proviene de una inserción manual.